DRAFT - Quick Start for Hierarchical Data

Center for Veterinary Biologics - Statistics Section

November 30, 2018

Contents

INTRODUCTION **2**
Data assumptions. Nested Hierarchical structure. 2
Mitigated Fraction. Bootstrapped and Calculated. 2
MF package basics. 2
Technical requirements. 2

CALCULATED MF **4**
General use. 4
Looking at MF for factors of a variable. 4
Common coding examples. 4
Basic Output. 5
Advanced usage. 6
Rank table. 6
Reproducibility. 7

BOOTSTRAPPED MF **8**
General use. 8
Common coding examples. 8
Basic output. 9
Advanced usage. 10
Bootstrapping step output. 10
Reproducibility. 12

APPENDIX **13**
Code for example data. 13
Session details for this manual. 13

INTRODUCTION

This document is intended to guide users through initial usage of functions in the `MF` package for evaluation of mitigated fraction when data is arranged in a nested hierarchical structure. Details of the algorithm are covered in a separate vignette.

Data assumptions. Nested Hierarchical structure.

Examples in this document expect data to be structured in a *nested hierarchical tree*, shown in Table 1. If there are only two variables (columns) or the experimental design is not nested, then the methods described here may not apply. A nested design assumes that the factor of one variable co-occur with another variable. For example, Pen D exists only within Room Z. If the experimental design of the data set is crossed (all factors of a variable co-occur with all factors of another variable), it is not appropriate to use this guide as written.

Mitigated Fraction. Bootstrapped and Calculated.

Users have the option of directly calculating the mitigated fraction or bootstrapping the mitigated fraction value. Examples of both options are shown in this document.

MF package basics.

Examples in this document make use of CVB-authored package `MF`, which can be found online at: https://github.com/ABS-dev/MF/blob/master/README.md, including installation instructions. It is expected that users have passing familiarity with R code and usage, as CVB does not have resources to address training or IT issues that may occur at external organizations.

Bug reports can be submitted online at: https://github.com/ABS-dev/MF/issues. To expedite resolution, please include a minimal working example and refrain from using confidential data. Incomplete issues may be closed without further investigation. Questions specific to a submission should be directed to the appropriate Reviewer.

Technical requirements.

Examples in this manual were created using MF version 4.3.5, R version 3.4.4 (2018-03-15) on Windows. CVB has not tested code usage on other systems.

Table 1: Nested hierarchical data structure.

room	pen	litter	tx	lung
Room W	Pen A	Litter 11	vac	5.633023
				4.618143
			con	9.196908
				7.277479
		Litter 12	vac	3.760431
				3.856939
			con	6.307358
				3.521230
	Pen B	Litter 13	vac	4.356031
				6.098324
			con	7.866934
				8.339948
		Litter 14	vac	5.389661
				5.799638
			con	8.275256
				7.952039
	Pen C	Litter 15	vac	4.855498
				5.657725
			con	7.658824
				8.517004
		Litter 16	vac	4.169034
				4.837650
			con	5.822156
				7.718757
Room Z	Pen D	Litter 17	vac	3.595465
				4.921762
			con	5.222559
				5.928738
		Litter 18	vac	7.622600
				5.036120
			con	8.620428
				6.265423
	Pen E	Litter 19	vac	3.787388
				5.380694
			con	9.454640
				6.505599
		Litter 20	vac	5.300123
				4.417709
			con	7.211453
				6.351508
	Pen F	Litter 21	vac	4.690320
				6.035818
			con	6.643916
				6.995050
		Litter 22	vac	4.896763
				5.371713
			con	6.773614

CALCULATED MF

General use.

To calculate mitigated fraction directly from data, use the function `MFClusHier`. This function requires four inputs:

- `formula`: The formula in form `y ~ x + a/b/c` where `y` is a continuous response, `x` is a factor with two levels of treatment, and `a/b/c` are variables corresponding to the clusters. It is expected that levels of "c" are nested within levels of "b". Nesting is assumed to be in order, left to right, highest to lowest.
- `data`: The data table (`data.frame` or `tibble`) with variables as identified in formula. If there are extra variables, they will be ignored.
- `compare`: A text vector of length two. The first position [1] is treated as the control or reference group two which members of the second position [2] are compared. If no value for the compare vector is supplied, it is assumed that the comparisons are between the `con` and `vac` treatment groups.
- `which.factor`: One or more variable(s) of interest. This can be any of the variables from the data set. If none or NULL is specified, MF will be calculated for the whole tree.

Looking at MF for factors of a variable.

The `which.factor` argument allows users the option of selecting if mitigated fraction should be calculated for factors of a particular variable. In the data from Table 1, it may be of interest to calculate the mitigated fraction for each factor of variable "room". It is possible to consider factors from multiple variables simultaneously, and the designator `All` can be used to evaluate for the entire tree, without breaking out by any variable. Evaluating the value of mitigated fraction for the entire tree is the default behavior, should the user not specify anything to the `which.factor` argument.

Common coding examples.

Default case, looking only at whole tree:

```
MFClusHier(formula = lung ~ tx + room/pen/litter, data = a)
```

Selecting a single variable to evaluate for each factor:

```
MFClusHier(formula = lung ~ tx + room/pen/litter, data = a,
                       which.factor = 'room')
```

Selecting multiple variables, including the entire tree:

```
MFClusHier(formula = lung ~ tx + room/pen/litter, data = a,
                       which.factor = c('room', 'litter', 'All'))
```

Basic Output.

The default display of `MFClusHier` output is a table with one row for each unique mitigated fraction calculated. A unique mitigated fraction value will be calculated for each factor of the variables identified by the user in the argument `which.factor` (see previous discussion). The total number of rows is sum of the number of unique factors for each variable the user specified.

```
mf_multiple <- MFClusHier(formula = lung ~ tx + room/pen/litter, data = a,
                       which.factor = c('room', 'litter', 'All'))
mf_multiple
```

```
## # A tibble: 15 x 9
##    variable level       MF  N1N2     U con_N vac_N con_medResp vac_medResp
##    <fct>    <chr>    <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>       <dbl>
##  1 room     Room W   0.833    24    22    12    12        7.79        4.85
##  2 room     Room Z   0.909    22    21    11    12        6.64        4.98
##  3 litter   Litter ~ 1         4     4     2     2        8.24        5.13
##  4 litter   Litter ~ 0         4     2     2     2        4.91        3.81
##  5 litter   Litter ~ 1         4     4     2     2        8.10        5.23
##  6 litter   Litter ~ 1         4     4     2     2        8.11        5.59
##  7 litter   Litter ~ 1         4     4     2     2        8.09        5.26
##  8 litter   Litter ~ 1         4     4     2     2        6.77        4.50
##  9 litter   Litter ~ 1         4     4     2     2        5.58        4.26
## 10 litter   Litter ~ 0.5       4     3     2     2        7.44        6.33
## 11 litter   Litter ~ 1         4     4     2     2        7.98        4.58
## 12 litter   Litter ~ 1         4     4     2     2        6.78        4.86
## 13 litter   Litter ~ 1         4     4     2     2        6.82        5.36
## 14 litter   Litter ~ 1         2     2     1     2        6.77        5.13
## 15 All      All      0.870    46    43    23    24        7.21        4.91
```

The variable columns are:

- `variable`: Which variable was considered when evaluating MF for the row.
- `level`: A unique factor level of the variable which was considered when evaluating MF for the row.
- `MF`: Mitigated fraction calculated value.
- `N1N2`: Sum of the `n1n2` values from the rank table for the factor level of that row.
- `U`: Sum of the `u` values from the rank table for the factor level of that row.
- `con_N` & `vac_N`: Sum of the counts from the rank table matching the particular factor level of the row. Note that the left hand side of the underscore will match values passed

by user to `compare`.

- `con_medResp` & `vac_medResp`: Median of responses for each comparison group matching the particular factor level of the row. Note that the left hand side of the underscore will match values passed by user to `compare`.

Advanced usage.

This section covers the technical description of how to access **MFh** and `MFnest()` function. For a full discussion of the algorithms of how **MFh**, `MFnest()` and `MFClusHier` are related, see [MF algorithms for hierarchical data].

Rank table.

To access the rank table, use the **MFh** field of the output object. For example:

```
thisMFh <- mf_multiple$MFh
thisMFh
```

```
## # A tibble: 12 x 10
##    room   pen   litter    con_medResp con_n     w vac_medResp vac_n  n1n2
##    <chr>  <chr> <chr>           <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl>
##  1 Room W Pen A Litter 11        8.24     2     7        5.13     2     4
##  2 Room W Pen A Litter 12        4.91     2     5        3.81     2     4
##  3 Room W Pen B Litter 13        8.10     2     7        5.23     2     4
##  4 Room W Pen B Litter 14        8.11     2     7        5.59     2     4
##  5 Room W Pen C Litter 15        8.09     2     7        5.26     2     4
##  6 Room W Pen C Litter 16        6.77     2     7        4.50     2     4
##  7 Room Z Pen D Litter 17        5.58     2     7        4.26     2     4
##  8 Room Z Pen D Litter 18        7.44     2     6        6.33     2     4
##  9 Room Z Pen E Litter 19        7.98     2     7        4.58     2     4
## 10 Room Z Pen E Litter 20        6.78     2     7        4.86     2     4
## 11 Room Z Pen F Litter 21        6.82     2     7        5.36     2     4
## 12 Room Z Pen F Litter 22        6.77     1     3        5.13     2     2
## # ... with 1 more variable: u <dbl>
```

This table includes the following information, used in the calculation of the basic output table described previously:

- `con_n` & `vac_n`: Counts of observations for each treatment for a particular instance of unique factor level of a variable. Note that the left hand side of the underscore will match values passed by user to `compare`.
- `n1n2`: Product of counts, `con_n` * `vac_n`.
- `w`: Wilcoxon statistic.
- `u`: Mann-Whitney statistic.

- `con_medResp` & `vac_medResp`: Median observation for for each treatment level in a particular instance of unique factor level of a variable. Note that the left hand side of the underscore will match values passed by user to `compare`.

There is one row for each unique combination of variable factor levels across the whole data set. This table shows the initial statistics as determined by the experimental design. Refer to the vignette for algorithm design [MF algorithms for hierarchical data] for in-depth discussion of how the rank is used to evaluate mitigated fraction values.

Reproducibility.

The rank table can be used to re-calculate the mitigated fraction values, for example if the user intends to explore a different selection to the `which.variable` argument. To do this, use the function `MFnest` which takes the following input arguments:

- `Y`: **MFh** field output as from `MFClusHier()`
- `which.factor`: As above.

For example:

```
MFnest(thisMFh, which.factor = "pen")
```

```
## # A tibble: 6 x 9
##   variable level    MF  N1N2     U con_N vac_N con_medResp vac_medResp
##   <fct>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>       <dbl>
## 1 pen      Pen A  0.5      8     6     4     4        6.79        4.24
## 2 pen      Pen B  1        8     8     4     4        8.11        5.59
## 3 pen      Pen C  1        8     8     4     4        7.69        4.85
## 4 pen      Pen D  0.75     8     7     4     4        6.10        4.98
## 5 pen      Pen E  1        8     8     4     4        6.86        4.86
## 6 pen      Pen F  1        6     6     3     4        6.77        5.13
```

This is the same output as if the user had initially selected for variable "pen":

```
MFClusHier(formula = lung ~ tx + room/pen/litter, data = a,
                      which.factor = "pen")
```

```
## # A tibble: 6 x 9
##   variable level    MF  N1N2     U con_N vac_N con_medResp vac_medResp
##   <fct>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>       <dbl>
## 1 pen      Pen A  0.5      8     6     4     4        6.79        4.24
## 2 pen      Pen B  1        8     8     4     4        8.11        5.59
## 3 pen      Pen C  1        8     8     4     4        7.69        4.85
## 4 pen      Pen D  0.75     8     7     4     4        6.10        4.98
## 5 pen      Pen E  1        8     8     4     4        6.86        4.86
## 6 pen      Pen F  1        6     6     3     4        6.77        5.13
```

BOOTSTRAPPED MF

General use.

The function `MFClusBootHier` allows for a bootstrapping approach to calculating mitigated fraction values. Input arguments are:

Same as in non-bootstrapped usage

- `formula`: The formula in form `y ~ x + a/b/c` where `y` is a continuous response, `x` is a factor with two levels of treatment, and `a/b/c` are variables corresponding to the clusters. It is expected that levels of "c" are nested within levels of "b". Nesting is assumed to be in order, left to right, highest to lowest.
- `data`: The data table (`data.frame` or `tibble`) with variables as identified in formula. If there are extra variables, they will be ignored.
- `compare`: A text vector of length two. The first position [1] is treated as the control or reference group two which members of the second position [2] are compared. If no value for the compare vector is supplied, it is assumed that the comparisons are between the `con` and `vac` treatment groups.
- `which.factor`: One or more variable(s) of interest. This can be any of the variables from the data set. If none or NULL is specified, MF will be calculated for the whole tree.

Additional arguments

- `nboot`: Number of bootstrapping events.
- `boot.unit`: Boolean whether to sample observations from within those of the same core.
- `boot.cluster`: Boolean whether to sample which cores are present. If TRUE, some trees have all the cores represented in the original data while others only have a subset.
- `alpha`: Used to calculate empirical hpd by shortest length interval.
- `seed`: Used to initialize random number generator for reproducibility.

A "core" is the unique combination of variable factors from the data, including the `compare` designation. In the example used in this document one core would be Room W/Pen A/Litter 11/vac and another would be Room Z/Pen F/Litter 22/con. Further discussion about the bootstrapping algorithm can be found in [MF algorithms for hierarchical data].

Common coding examples.

Default case, looking only at whole tree and bootstrapping both at the cluster and unit levels:

```
MFClusBootHier(formula = lung ~ tx + room/pen/litter,
                              data = a)
```

Specifying what bootstrapping sampling to occur:

```r
MFClusBootHier(formula = lung ~ tx + room/pen/litter,
                                data = a, boot.unit = TRUE,
                                boot.cluster = FALSE, which.factor = 'room')
```

Adjusting the number of bootstrapping events and alpha:

```r
MFClusBootHier(formula = lung ~ tx + room/pen/litter,
                                    data = a, boot.unit = FALSE,
                                    boot.cluster = TRUE,  alpha = 0.1,
                                    which.factor = c('room', 'litter', 'All'))
```

Basic output.

The default display of `MFClusBootHier` is a table with one row for each unique mitigated fraction calculated, just like in the non-bootstrapped approach. However, instead of summary statistics about the calculated mitigated fraction, there are values summarizing the mitigated fraction from a bootstrapped population.

```r
mfboot_multiple <- MFClusBootHier(formula = lung ~ tx + room/pen/litter,
                                    data = a, boot.unit = FALSE,
                                    boot.cluster = TRUE,  alpha = 0.1,
                                    which.factor = c('room', 'litter', 'All'))
mfboot_multiple
```

```
## # A tibble: 15 x 8
##    variable level     median etlower etupper htlower htupper mf.obs
##    <fct>    <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>  <dbl>
##  1 room     Room W     0.857   0.5       1      0.6      1    0.833
##  2 room     Room Z     0.917   0.75      1      0.8      1    0.909
##  3 litter   Litter 11  1       1         1      1        1    1
##  4 litter   Litter 12  0       0         0      0        0    0
##  5 litter   Litter 13  1       1         1      1        1    1
##  6 litter   Litter 14  1       1         1      1        1    1
##  7 litter   Litter 15  1       1         1      1        1    1
##  8 litter   Litter 16  1       1         1      1        1    1
##  9 litter   Litter 17  1       1         1      1        1    1
## 10 litter   Litter 18  0.5     0.5       0.5    0.5      0.5  0.5
## 11 litter   Litter 19  1       1         1      1        1    1
## 12 litter   Litter 20  1       1         1      1        1    1
## 13 litter   Litter 21  1       1         1      1        1    1
## 14 litter   Litter 22  1       1         1      1        1    1
## 15 All      All        0.875   0.708     1      0.75     1    0.870
```

The variable columns are:

- `variable`: Which variable was considered when evaluating MF for the row.
- `level`: A unique factor level of the variable which was considered when evaluating MF for the row.
- `median`: Median of mitigated fractions calculated from the bootstrapped population.
- `etlower`: Lower value of equal tailed range of mitigated fractions calculated from the bootstrapped population.
- `etupper`: Upper value of equal tailed range of mitigated fractions calculated from the bootstrapped population.
- `htlower`: Lower value of the high tailed range of mitigated fractions calculated from the bootstrapped population.
- `htupper`: Upper value of the high tailed range of mitigated fractions calculated from the bootstrapped population.
- `mf.obs`: Mitigated fraction value calculated from data input, without bootstrapping.

Advanced usage.

This section covers the technical description of how to access **MFhBoot** and `MFnestBoot()` function. For a full discussion of the algorithms of how **MFhBoot**, `MFnestBoot()` and `MFClusBootHier` are related, or details of the bootstrapping algorithm, see [MF algorithms for hierarchical data].

Bootstrapping step output.

The bootstrapping stage is the most computationally intensive, so a user may wish to bypass this step subsequently if the only change is what values are being passed to the `which.factor` argument. Access bootstrapping step outputs using the **MFhBoot** field of the output object. For example:

```
thisBootMFh <- mfboot_multiple$MFhBoot
thisBootMFh
```

```
## $bootmfh
## # A tibble: 120,000 x 11
##    bootID con_medResp con_n     w vac_medResp vac_n     u  n1n2 room
##     <int>       <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl> <dbl> <chr>
##  1      1        4.91     2     5        3.81     2     2     4 Room W
##  2      1        4.91     2     5        3.81     2     2     4 Room W
##  3      1        8.11     2     7        5.59     2     4     4 Room W
##  4      1        8.11     2     7        5.59     2     4     4 Room W
##  5      1        8.11     2     7        5.59     2     4     4 Room W
##  6      1        8.09     2     7        5.26     2     4     4 Room W
##  7      1        8.09     2     7        5.26     2     4     4 Room W
```

```
## 8      1        6.77     2     7        4.50     2     4     4 Room W
## 9      1        5.58     2     7        4.26     2     4     4 Room Z
## 10     1        6.82     2     7        5.36     2     4     4 Room Z
## # ... with 119,990 more rows, and 2 more variables: pen <chr>,
## #   litter <chr>
##
## $clusters
##      room   pen    litter clusterID
## 1  Room W Pen A Litter 11         1
## 2  Room W Pen A Litter 12         2
## 3  Room W Pen B Litter 13         3
## 4  Room W Pen B Litter 14         4
## 5  Room W Pen C Litter 15         5
## 6  Room W Pen C Litter 16         6
## 7  Room Z Pen D Litter 17         7
## 8  Room Z Pen D Litter 18         8
## 9  Room Z Pen E Litter 19         9
## 10 Room Z Pen E Litter 20        10
## 11 Room Z Pen F Litter 21        11
## 12 Room Z Pen F Litter 22        12
##
## $compare
## [1] "con" "vac"
##
## $mfh
## # A tibble: 12 x 10
##    room   pen   litter    con_medResp con_n     w vac_medResp vac_n  n1n2
##    <chr>  <chr> <chr>           <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl>
##  1 Room W Pen A Litter 11        8.24     2     7        5.13     2     4
##  2 Room W Pen A Litter 12        4.91     2     5        3.81     2     4
##  3 Room W Pen B Litter 13        8.10     2     7        5.23     2     4
##  4 Room W Pen B Litter 14        8.11     2     7        5.59     2     4
##  5 Room W Pen C Litter 15        8.09     2     7        5.26     2     4
##  6 Room W Pen C Litter 16        6.77     2     7        4.50     2     4
##  7 Room Z Pen D Litter 17        5.58     2     7        4.26     2     4
##  8 Room Z Pen D Litter 18        7.44     2     6        6.33     2     4
##  9 Room Z Pen E Litter 19        7.98     2     7        4.58     2     4
## 10 Room Z Pen E Litter 20        6.78     2     7        4.86     2     4
## 11 Room Z Pen F Litter 21        6.82     2     7        5.36     2     4
## 12 Room Z Pen F Litter 22        6.77     1     3        5.13     2     2
## # ... with 1 more variable: u <dbl>
```

This output is a list of four objects:

- `bootmfh`: The rank table for each of the bootstrapped data sets. Formatted as the rank table for non-bootstrapped data with designation of bootstrapping instance.

- `clusters`: Unique cores in the data.
- `compare`: User-supplied value passed to compare argument.
- `mfh`: The rank table from user-supplied data. Formatted as for non-bootstrapped data.

Reproducibility.

To bootstrap from the same data in a reproducible manner, numerical value for the **seed** argument in `MFClusBootHier`. If no value for `seed` is specified, the function defaults to a value of `sample(1:1e+05, 1)` and subsequent iterations may yield different summaries.

```
MFClusBootHier(formula = lung ~ tx + room/pen/litter,
                              data = a, seed = 150)
```

Since the bootstrapping step is the most computationally intensive, a user may wish to use the bootstrapped rank table to recalculate mitigated fraction for factors of different variables or a different `alpha` for calculating confidence intervals. This is possible using the `MFnestBoot` function, which takes the following input arguments:

- `x`: **MFhBoot** field output as from `MFClusBootHier`.
- `which.factor`: As above.
- `alpha`: As above.

For example:

```
MFnestBoot(thisBootMFh, which.factor = c('pen', 'All'), alpha = 0.1)
```

```
## Complete separation observed for variable(s): pen

## $mfnest_details
## # A tibble: 63,292 x 8
## # Groups:   variable, level [?]
##    variable level bootID     U  N1N2 con_N vac_N    MF
##    <chr>    <chr>  <int> <dbl> <dbl> <dbl> <dbl> <dbl>
##  1 All      All        1    42    46    23    24 0.826
##  2 All      All        2    45    48    24    24 0.875
##  3 All      All        3    42    46    23    24 0.826
##  4 All      All        4    42    46    23    24 0.826
##  5 All      All        5    40    44    22    24 0.818
##  6 All      All        6    44    46    23    24 0.913
##  7 All      All        7    44    44    22    24 1
##  8 All      All        8    46    46    23    24 1
##  9 All      All        9    44    46    23    24 0.913
## 10 All      All       10    42    42    21    24 1
## # ... with 63,282 more rows
##
## $mfnest_summary
## # A tibble: 7 x 8
```

```
##   variable level median etlower etupper htlower htupper mf.obs
##   <fct>    <chr>  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>  <dbl>
## 1 pen      Pen A  0.5     0           1    0          1  0.5
## 2 pen      Pen B  1       1           1    1          1  1
## 3 pen      Pen C  1       1           1    1          1  1
## 4 pen      Pen D  0.75    0.5         1    0.5        1  0.75
## 5 pen      Pen E  1       1           1    1          1  1
## 6 pen      Pen F  1       1           1    1          1  1
## 7 All      All    0.875   0.708       1    0.75       1  0.870
```

APPENDIX

Code for example data.

```r
a <- data.frame(
 room = paste('Room', rep(c('W', 'Z'), each = 24)),
 pen = paste('Pen', rep(LETTERS[1:6], each = 8)),
 litter = paste('Litter', rep(11:22, each = 4)),
 tx = rep(rep(c('vac', 'con'), each = 2), 12),
 stringsAsFactors = FALSE
 )
set.seed(76153)
a$lung[a$tx == 'vac'] <- rnorm(24, 5, 1.3)
a$lung[a$tx == 'con'] <- rnorm(24, 7, 1.3)
a <- a[-48,]
```

Session details for this manual.

```r
sessionInfo()
```

```
## R version 3.4.4 (2018-03-15)
## Platform: x86_64-w64-mingw32/x64 (64-bit)
## Running under: Windows 10 x64 (build 14393)
##
## Matrix products: default
##
## locale:
## [1] LC_COLLATE=English_United States.1252
## [2] LC_CTYPE=English_United States.1252
## [3] LC_MONETARY=English_United States.1252
## [4] LC_NUMERIC=C
```

```
## [5] LC_TIME=English_United States.1252
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] bindrcpp_0.2.2   MF_4.3.5         forcats_0.3.0    stringr_1.3.1
##  [5] dplyr_0.7.8      purrr_0.2.5      readr_1.1.1      tidyr_0.8.2
##  [9] tibble_1.4.2     ggplot2_3.0.0    tidyverse_1.2.1  kableExtra_0.9.0
##
## loaded via a namespace (and not attached):
##  [1] Rcpp_1.0.0        cellranger_1.1.0  pillar_1.2.3
##  [4] compiler_3.4.4    plyr_1.8.4        bindr_0.1.1
##  [7] tools_3.4.4       digest_0.6.15     lubridate_1.7.4
## [10] jsonlite_1.5      gtable_0.2.0      evaluate_0.10.1
## [13] nlme_3.1-131.1    viridisLite_0.3.0 lattice_0.20-35
## [16] pkgconfig_2.0.1   rlang_0.3.0.1     cli_1.0.0
## [19] rstudioapi_0.7    yaml_2.1.19       haven_1.1.2
## [22] withr_2.1.2       httr_1.3.1        xml2_1.2.0
## [25] knitr_1.20        hms_0.4.2         grid_3.4.4
## [28] rprojroot_1.3-2   tidyselect_0.2.5  glue_1.3.0
## [31] R6_2.2.2          readxl_1.1.0      rmarkdown_1.10
## [34] modelr_0.1.2      magrittr_1.5      backports_1.1.2
## [37] scales_0.5.0      htmltools_0.3.6   assertthat_0.2.0
## [40] rvest_0.3.2       colorspace_1.3-2  utf8_1.1.4
## [43] stringi_1.1.7     lazyeval_0.2.1    munsell_0.5.0
## [46] broom_0.5.0       crayon_1.3.4
```